# You Don't Need Jev

Cheap LLM Labels and a 150M-Parameter Encoder on a CPU
Match or Beat a Hosted Decision Model on Accuracy and Calibration,
at a Fraction of the Cost and Latency

Erfan Eshratifar
me@amirerfan.com

Code and data: https://github.com/erfaneshrati/ownyourjev

October 2026

**Abstract**

Hosted "decision models" such as TypeSafe's Jev answer typed questions about text through a paid API: yes/no, pick one of $k$ options, or score on an ordered rubric. Their appeal rests on four claims: accurate answers without training, calibrated probabilities, low latency, and low cost. We show that a practitioner can match or beat such a model on all four with a model they own, built in an afternoon for a few dollars. Our recipe uses a cheap LLM to label the practitioner's own unlabeled text, adds a budget of only 220 human-labeled examples per task, distils the result into a ∼150M-parameter encoder, and serves it with ONNX Runtime on a CPU. We evaluate on the 12 classification and similarity tasks of a published Jev benchmark, scored on exactly the same test examples as Jev.

- **Accuracy.** The owned models win on 8 tasks (95% bootstrap CI entirely above Jev) and tie on the other 4. They lose on none.
- **Calibration.** A calibrator fitted on the same 200 dev examples gives lower expected calibration error than Jev on 12 of 14 question heads (median 0.027 vs. 0.086), and lower log-loss on 12 of 14.
- **Latency.** They answer in 12–30 ms at the median and at most 68 ms at p95 on four CPU threads, below Jev's best published end-to-end latency of 70 ms.
- **Cost.** They cost 12–84× less per decision. The full experimental spend for each task, $0.15–$2.57 including every failed attempt, is recovered after 4k–181k decisions.

To make these claims credible we use a strict protocol. All choices are made on a 200-example dev set drawn from the training split, test labels are read only by audited scripts, and every one of the 14 test scorings is reported. We release the toolkit (`ownyourjev`) with a Jev-compatible local server, per-task ledgers of 318 recorded experiments including failures, and all evaluation code at https://github.com/erfaneshrati/ownyourjev. The largest single lever was not model size but the labeling prompt: guidelines written from 20 examples and from the labeler's mistakes raised labeler agreement with gold by 6–30 points.

## 1 Introduction

Many production systems need fast, typed judgments about text: route this ticket, is this message spam, how similar are these two sentences. *Decision models* package this as a service. Jev, the first "System One" model from TypeSafe AI [18], takes a state and a set of questions and returns calibrated answers of three types: *Noul* (a probability that a proposition is true), *Choice* (a distribution over unordered options), and *Score* (a distribution over an ordered rubric and its expected

| | Jev (hosted) | Owned model (CPU) |
|---|---|---|
| Accuracy, 12 tasks | – | **8 wins, 4 ties, 0 losses** |
| Calibration error (ECE), 14 heads | median 0.086 | **median 0.027**; lower on 12/14 |
| Latency per request | 70–500 ms | **12–30 ms p50, ≤68 ms p95** |
| Cost per 1M decisions | $13–$61 | **$0.52–$1.34** |
| One-time build cost | – | $0.15–$2.57 per task |
| Human labels | none | 220 per task |
| Data leaves your machine | yes | **no** |
| Behavior changes without notice | possible | **no** |

Table 1: Scorecard. Jev numbers are `jev-1.13.0` on the same test examples; its latency is the published end-to-end range. Our latency is per request at batch size 1 on four threads of an Intel i9-10900X; build cost includes every failed experiment. Details in Section 5.

value). It is cheap per call ($0.042 per million input tokens), returns in 70–500 ms, and a recent independent evaluation found it competitive with open LLMs on a broad suite of public datasets [5].

A hosted model also brings an external dependency, per-call cost that grows with traffic, network latency, data leaving the premises, and behavior that can change between versions. The alternative is old and simple: train a small classifier for each decision. Its traditional obstacle is labeled data. That obstacle has largely disappeared, because LLMs annotate text well and cheaply [7, 19], and a small encoder can learn from those annotations [10].

Jev's value proposition rests on four properties: it is accurate without any training, its probabilities are calibrated, it is fast, and it is cheap. We test whether a model a practitioner owns can beat it on each of these, on Jev's own benchmark, under realistic constraints:

1. a human-label budget of only 220 examples per task;
2. inference on a commodity CPU, faster than Jev;
3. a total LLM labeling budget measured in dollars, not hundreds of dollars;
4. an evaluation protocol strict enough that the comparison can be trusted.

Table 1 summarizes the answer: on every axis Jev competes on, the owned model matches or beats it on most tasks, and loses on none in accuracy.

**Contributions.**

- A head-to-head comparison with Jev on 12 tasks (4 Noul, 5 Choice, 3 Score), on exactly the examples Jev was scored on: 8 wins, 4 ties and no losses (Table 2, Figure 1).
- Cost and latency accounting: 12–84× cheaper per decision; p95 latency of 19–68 ms on four CPU threads (Table 3, Figure 2).
- An evaluation protocol designed against test-set overfitting, including a fixed human-label budget, dev-only selection, an audited and logged test reader, and dev-gated re-scoring. We report every test look.
- Calibration: with a calibrator fitted on the same 200 dev examples, our probabilities are better calibrated than Jev's on 12 of 14 heads. On half of them they beat even Jev recalibrated on the test labels (Table 4, Figure 3).
- An open-source toolkit, `ownyourjev` (`https://github.com/erfaneshrati/ownyourjev`), that turns a Jev question specification into a calibrated local model behind a Jev-compatible API, released with all experiment ledgers and evaluation code.
- A practitioner's recipe distilled from 318 logged experiments, and an account of what did *not* help: larger models, frontier labelers on noisy tasks, and synthetic data that does not match the

real distribution.

## 2 Related work

**Decision models and Jev.** Deußer et al. [5] benchmark `jev-1.13.0` zero-shot on 37 public datasets and release their question templates, splits, and the raw per-example responses. We reuse all three, which lets us compare with Jev on identical inputs without calling its API.

**LLMs as annotators.** LLM labels can replace or augment crowd labels at a fraction of the cost [7, 19]. We find that the *prompt* given to the annotator matters more than the annotator's size, and that a student trained on LLM labels is bounded by, and sometimes below, the labels' agreement with gold.

**Small encoders, distillation and efficient inference.** We use modern 150M-parameter encoders: ModernBERT-based GTE [12, 20] and UltraBERT [6]. We train them with soft-label distillation [10] and weight averaging [21], calibrate them with temperature scaling [8], and serve them with ONNX Runtime [15] using int8 quantization with SmoothQuant-style rescaling [22].

## 3 Experimental setup

### 3.1 Tasks and the Jev baseline

We use 12 tasks from the benchmark of Deußer et al. [5], with their splits, metrics and question templates:

- **Noul:** SMS Spam [1], PAWS [24], Prompt Injections [4], and ToxicChat [13]. ToxicChat asks two questions per request: toxic and jailbreak.
- **Choice:** Banking77 [2], CLINC150 with out-of-scope [11], AG News [23], Financial PhraseBank [14], and Emotion [16].
- **Score:** STS-B [3], SST-5 [17], and ToxiGen [9]. ToxiGen asks a Noul toxicity question and a 5-level Score question in one request.

The benchmark also contains reading comprehension and the judging of generated text (BoolQ, SummEval, HelpSteer2). These are left to future work and are not reported here.

Jev numbers are computed from the published per-example `jev-1.13.0` responses on exactly our test examples. They match the published figures except for SMS Spam, where 220 examples are carved out of the only split (Section 3.2).

### 3.2 Protocol

An earlier exploratory round of this project chose variants after seeing test scores. We discarded its results and fixed the following protocol *before* any reported experiment.

- **Human-label budget.** Each task gets 20 *seed* examples and 200 *dev* examples with gold labels, drawn from the training split with a fixed seed. All other gold labels are hidden from experiments: the training pool and the test inputs are exposed as unlabeled text.
- **Tasks without a training split.** SMS Spam has no training split, so its 220 examples are carved out of the evaluation set, Jev is re-scored on the remainder, and models are cross-fitted. Every text is scored by a model that never trained on it.
- **Selection on dev only.** Every decision is made on dev: labeler, prompt, label cleaning, data mix, backbone, epochs, threshold, and quantization.

- **Audited test access.** A single script reads test labels. It requires a frozen recipe file written beforehand, and it logs every call.
- **Dev-gated re-scoring.** A task may be scored on test again only if a new recipe beats the previous one on dev by more than dev noise (a paired bootstrap CI excluding zero, or a seed-mean comparison), with a logged reason. Earlier results are archived, and all 14 test looks are reported (Table 2).
- **Latency gate.** A model qualifies only if its CPU p95 latency per request, with batch size 1, 4 threads, and all of a task's questions answered, is below 70 ms. That is the bottom of Jev's published end-to-end range.
- **Budget.** Backbones are limited to ∼150M parameters. Total LLM spend for the whole project was capped at $25.

### 3.3 Infrastructure

Experiments were executed in parallel by autonomous coding agents, one per task, all following the same written brief. A coordinator enforced per-task spend caps, scheduled the shared GPUs, and audited test access. Every idea an agent tried was written to a per-task ledger with its dev evidence, cost and outcome, giving 318 entries for the 12 tasks. Training used an RTX 4090 and an RTX 3080. All reported latencies were re-measured on an idle Intel i9-10900X (10 cores), using ONNX Runtime 1.31. LLM labels came from OpenRouter, mostly from `deepseek-v4.1-flash` ($0.057/$0.144 per million input/output tokens) and `qwen3.7-flash`.

## 4 Method: owning the decision

The `ownyourjev` pipeline mirrors Jev's interface. A user writes the same question specification they would send to Jev. The system then produces a model that answers it locally, behind a Jev-compatible `/v1/systemone` endpoint.

**1. Label the pool with an LLM, guided.** The labeling prompt is built from the question specification and adds three things:
- *Labeling guidelines,* written from the 20 seeds and from the labeler's errors on one half of dev, then checked on the other half.
- The seeds as few-shot examples.
- Optionally, a request for a probability per option (soft labels).

Batching 10–50 items per call amortizes the long prompt and cut cost 10–20× without loss of agreement on most tasks. Several cheap labelers can be averaged into soft targets.

**2. Use the whole pool, and add data only where it is missing.** We label the entire training pool. Synthetic data is added only to fill a gap the real data does not cover:
- out-of-scope utterances for CLINC150;
- hard positives and negatives for ToxicChat;
- seed-steered injections for Prompt Injections;
- re-paired sentence pairs spanning the similarity scale for STS-B.

**3. Train a small student.** A 150M encoder is fine-tuned with one linear head per question, or one shared encoder with several heads. The heads are a softmax over options or levels for Choice and Score, and a sigmoid for Noul; the Score head reports the expected level. Targets are soft

| Type | Task | Metric | $n_{\text{test}}$ | Ours [95% CI] | Jev | Verdict | Test looks |
|---|---|---|---|---|---|---|---|
| Noul | SMS Spam | F1 | 5,354 | **0.958** [0.946, 0.969] | 0.937 | win | 1 |
| Noul | PAWS | Acc. | 8,000 | **0.884** [0.877, 0.891] | 0.850 | win | 1 |
| Noul | Prompt Injections | Acc. | 116 | **0.957** [0.922, 0.991] | 0.741 | win | 1 |
| Noul | ToxicChat | F1 | 5,083 | 0.787 [0.755, 0.818] | 0.786 | tie | 2 |
| Choice | Banking77 | Acc. | 3,076 | **0.861** [0.849, 0.873] | 0.797 | win | 1 |
| Choice | CLINC150 | Acc. | 5,500 | **0.934** [0.927, 0.941] | 0.895 | win | 1 |
| Choice | AG News | Acc. | 7,600 | **0.893** [0.886, 0.900] | 0.885 | win | 1 |
| Choice | Financial PhraseBank | Acc. | 970 | **0.794** [0.770, 0.819] | 0.731 | win | 1 |
| Choice | Emotion | Acc. | 2,000 | **0.811** [0.794, 0.829] | 0.585 | win | 1 |
| Score | STS-B | Spearman | 1,379 | 0.893 [0.881, 0.905] | 0.890 | tie | 2 |
| Score | SST-5 | Acc. | 2,210 | 0.583 [0.562, 0.603] | 0.579 | tie | 1 |
| Score | ToxiGen | Acc. | 940 | 0.854 [0.830, 0.878] | 0.878 | tie | 1 |

Table 2: Main results. Ours: a ∼150M-parameter encoder trained on LLM labels plus a 220-example human-label budget, served as ONNX on CPU. Jev: `jev-1.13.0` re-scored from its published per-example responses on exactly the same test examples. Verdict: *win* if our 95% bootstrap CI lies entirely above Jev, *tie* if it contains Jev. *Test looks* counts how often the test split was scored for the task; second looks were permitted only after a dev-gated improvement (Section 3.2). For ToxicChat the headline is the toxic head; its jailbreak head scores 0.848 vs. Jev 0.717. For ToxiGen the second (toxicity) head scores Spearman 0.790 vs. 0.840.

where available. A shared-initialization weight soup removes most seed variance at no inference cost. Decision thresholds are fitted on held-out labels or on dev.

**4. Calibrate on dev.** The 200 dev examples double as calibration data. `ownyourjev calibrate` predicts them one request at a time, as the model will be served, and picks a calibrator by 5-fold cross-validated log-loss. Choice and Score heads may use none or temperature scaling, which cannot change the chosen option; Noul heads may also use Platt scaling or isotonic regression. The calibrator is saved next to the model, and the predictor and the Jev-compatible server apply it automatically.

**5. Export for CPU.** We export to ONNX and quantize to int8. Two details proved essential:

- Keeping the MLP output projections in fp32, or rescaling their input activations SmoothQuant-style, recovers fp32 accuracy at near-int8 speed. Full int8 collapsed UltraBERT entirely.
- Exporting attention in eager form avoids per-layer masking ops that cost 15–18% of CPU time.

# 5 Results

**Accuracy.** Table 2 and Figure 1 show the results. The owned models beat Jev on 8 of 12 tasks, with the 95% CI entirely above Jev.

- **Large margins** where Jev's zero-shot reading of the question misses the dataset's labeling convention: Emotion +0.227 and Prompt Injections +0.216.
- **Clear margins** on routing and domain sentiment: Banking77 +0.063, Financial PhraseBank +0.063, CLINC150 +0.039 and PAWS +0.035.
- **Narrow but clean wins** on SMS Spam (+0.021) and AG News (+0.008).

Four tasks are ties:

- STS-B, SST-5 and ToxicChat (toxic head) are within 0.004 of Jev.

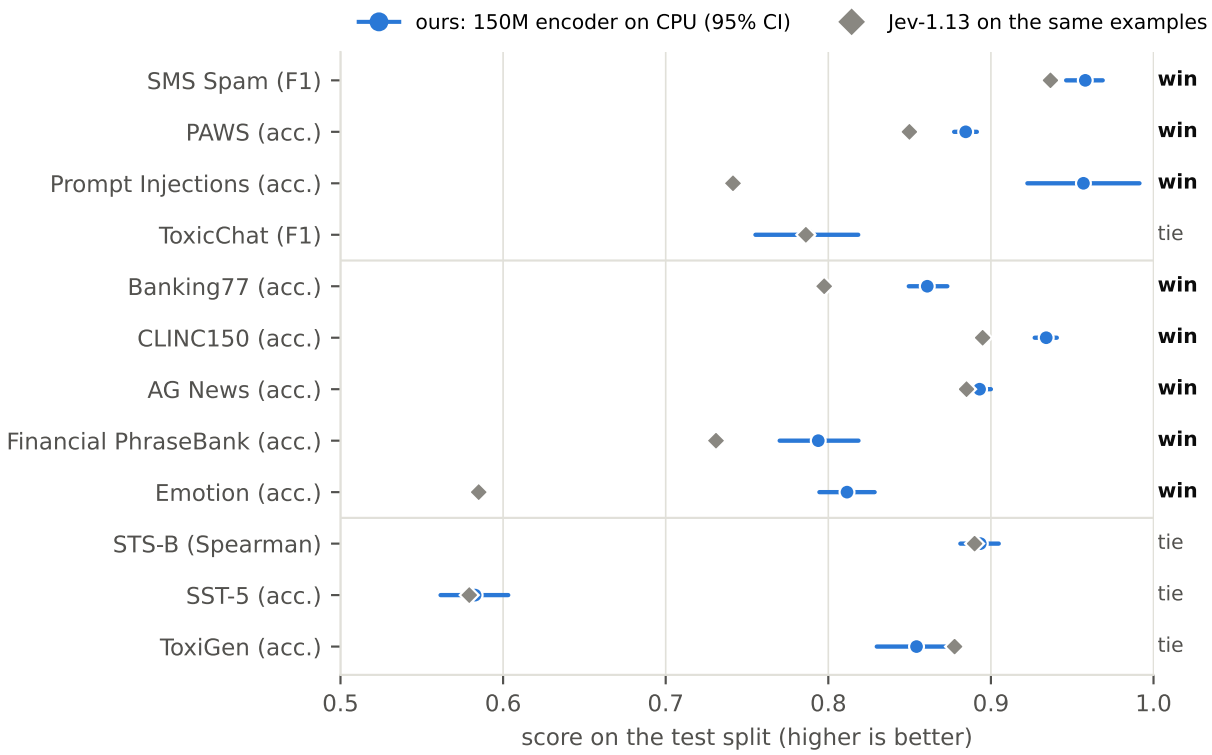



Figure 1: Test-set scores with 95% bootstrap CIs (ours) and Jev on the same examples. Horizontal rules separate the Noul, Choice and Score tasks.

- On ToxicChat, the second question asked in the same request (jailbreak) is a clear win, 0.848 vs. 0.717.
- On ToxiGen our point estimate is 0.023 below Jev, with a CI that just reaches it.

In no task is our CI entirely below Jev.

**Cost and latency.** Table 3 and Figure 2 show the cost and latency results.

- **Cost per decision:** Jev costs \$13–\$61 per million decisions, depending on prompt length. A 150M int8 encoder on four CPU threads costs \$0.52–\$1.34 at \$0.04 per vCPU-hour, which is 12–84× cheaper.
- **Build cost:** including every exploratory and failed experiment, it ranged from \$0.15 to \$2.57 per task (\$12.53 for all 12). It is recovered after 4k–181k decisions.
- **Latency:** median latency is 12–30 ms and p95 is 19–68 ms, below Jev's published floor. Jev's figure includes network time and ours does not; that is part of the argument for owning the model.

**Calibration.** Jev's selling point is calibrated probabilities, so a replacement must return probabilities that can be trusted, not just correct decisions. We ran a separate post-hoc calibration study on the frozen models.

- **Fitting.** For each head, the deployed model predicted the 200 gold dev examples out of sample. Where a model had trained on dev or seeds (cross-fitted SMS Spam, Prompt Injections), we used out-of-fold predictions. Candidate calibrators are none, temperature [8], Platt scaling and

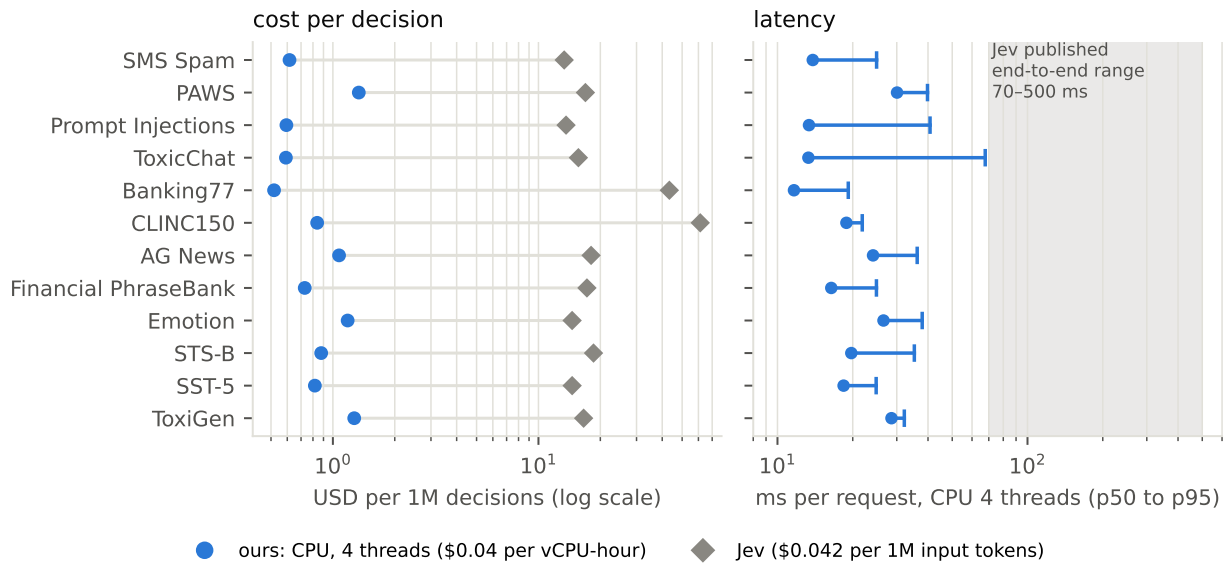



Figure 2: Left: cost per million decisions (log scale). Right: our CPU latency (dot = p50, bar end = p95) against Jev's published 70–500 ms end-to-end range.

isotonic regression, chosen by 5-fold cross-validated NLL on dev and refitted on all of dev.

- **Decisions never change.** Choice and Score heads may only use argmax-preserving calibrators (none or temperature). Noul decisions stay at the frozen threshold on the raw probability. Accuracy and every verdict in Table 2 are therefore unaffected.
- **One look.** Each calibrator was frozen before a single audited, logged test scoring.
- **Oracle rows.** As a like-for-like upper bound, we also recalibrated each system by 5-fold CV on the test labels with the same allowed methods. No deployable system could do this; it measures how much of each system's miscalibration is fixable.

Table 4 and Figure 3 show the results.

- With a calibrator fitted on just 200 dev examples, our expected calibration error is lower than Jev's on 12 of 14 heads (median 0.027 vs. 0.086). NLL is lower on 12 of 14 and Brier score on 10 of 14.
- Jev is badly overconfident on Emotion (ECE 0.279, NLL 2.83), Prompt Injections (0.169) and SST-5 (0.170). On 7 of 14 heads our dev-calibrated model beats even the test-oracle recalibration of Jev in ECE, and on 9 of 14 in Brier and NLL.
- Jev stays better calibrated on three heads: ToxiGen's toxic head (ECE 0.014 vs. 0.032; Jev also has the higher AUROC there), ToxicChat's toxic head (Brier and NLL only; our ECE is lower), and CLINC150.
- On CLINC150 our out-of-scope rule sets the out-of-scope probability just above the best in-scope class. On the 18% of inputs where it fires, the reported top probability is therefore about 0.28 although these predictions are mostly correct. The 200 dev examples contain only 8 out-of-scope cases, so dev selection could not detect this and kept the raw probabilities. Emitting a proper out-of-scope probability would fix it, but would change the deployed output and is left for future work.
- Our raw probabilities err in both directions. They are overconfident on Emotion, AG News, SST-5 and STS-B (fitted $T$ of 1.2–1.6) and underconfident on Banking77 and both ToxicChat heads ($T$ of 0.4–0.85). In every case a single parameter fitted on 200 dev examples removes most

| | CPU ms | | Build | USD / 1M dec. | | | Break- |
|---|---|---|---|---|---|---|---|
| Task | p50 | p95 | cost | Jev | Ours | Ratio | even |
| SMS Spam | 14 | 25 | $0.23 | $13.34 | $0.62 | 22× | 18k |
| PAWS | 30 | 40 | $0.31 | $16.92 | $1.34 | 13× | 20k |
| Prompt Injections | 13 | 41 | $0.19 | $13.62 | $0.59 | 23× | 14k |
| ToxicChat | 13 | 68 | $2.27 | $15.64 | $0.59 | 26× | 151k |
| Banking77 | 12 | 19 | $0.92 | $43.33 | $0.52 | 84× | 21k |
| CLINC150 | 19 | 22 | $0.24 | $61.24 | $0.84 | 73× | 4k |
| AG News | 24 | 36 | $0.26 | $18.03 | $1.07 | 17× | 15k |
| Financial PhraseBank | 16 | 25 | $0.15 | $17.21 | $0.73 | 24× | 9k |
| Emotion | 27 | 38 | $0.38 | $14.58 | $1.18 | 12× | 28k |
| STS-B | 20 | 35 | $2.57 | $18.53 | $0.88 | 21× | 146k |
| SST-5 | 18 | 25 | $2.49 | $14.58 | $0.82 | 18× | 181k |
| ToxiGen | 29 | 32 | $2.53 | $16.59 | $1.27 | 13× | 165k |

Table 3: Latency and cost. CPU latency is per request (all of a task's questions), batch 1, 4 threads, worst of repeated runs on an idle Intel i9-10900X; Jev's published end-to-end latency is 70–500 ms. *Build cost* is the total OpenRouter spend for the task including every exploratory and failed experiment, so it is an upper bound on what a practitioner would pay. Jev cost uses its measured input tokens on these examples at $0.042 per 1M input tokens; ours assumes $0.04 per vCPU-hour. *Break-even* is the number of decisions after which owning the model is cheaper.

of the error.

Two rules were fixed during the study and applied to all tasks. First, the argmax-preserving restriction: two tasks had initially selected vector scaling, which can change the top choice. AG News was re-frozen before scoring. STS-B was re-scored once under a logged reason; its vector-scaling result is kept in the repository. ToxiGen's toxicity head had been scored with vector scaling, and its compliant dev choice, *none*, is the raw row, so no re-score was needed. Second, the oracle rows were recomputed centrally with the same restriction, because the first version could re-classify. Both changes are recorded in the test-access log.

**Serving consistency at batch size 1.** During the calibration study, re-running the int8 models on dev did not always reproduce their frozen test probabilities. The cause is dynamic int8 quantization: it computes activation scales over the whole input tensor, padding included, so a text's probabilities depend on the other texts in its batch. Eight int8 tasks had computed their frozen test predictions in batches, while the model is served, and its latency measured, one request at a time. We recomputed those predictions at batch size 1 with the same artifacts and scored them once through a separate audited script (Table 5).

- Every verdict is unchanged. The headline metric moves by −0.003 to +0.024, and the batch-1 outputs are deterministic.
- Decision flips are rare on most tasks (0.3–1.5%) but reach 3.7% on Banking77, 5.1% on SST-5 and 9.5% on Financial PhraseBank. There, int8 decisions near the neutral/positive boundary are fragile, and the gain at batch 1 (0.794 to 0.818 on test) is not matched on dev (0.850 to 0.830), so it should be read as noise.
- Table 2 keeps the pre-registered batched numbers. The calibration results above use the batch-1 predictions, with calibrators refitted on batch-1 dev predictions.
- ToxicChat and STS-B were already predicted at batch size 1. Emotion and ToxiGen run in fp32, where batching only adds floating-point noise.

| | | ECE | | | Brier | | NLL | | ECE, oracle | |
|---|---|---|---|---|---|---|---|---|---|---|
| Task (head) | calibrator | ours raw | ours | Jev | ours | Jev | ours | Jev | ours | Jev |
| SMS Spam | – | 0.006 | **0.006** | 0.031 | **0.009** | 0.015 | **0.039** | 0.072 | 0.002 | 0.002 |
| PAWS | Platt | 0.051 | **0.015** | 0.020 | **0.082** | 0.109 | **0.269** | 0.348 | 0.006 | 0.005 |
| Prompt Injections | Platt | 0.023 | **0.022** | 0.169 | **0.026** | 0.192 | **0.098** | 0.590 | 0.041 | 0.067 |
| ToxicChat (toxic) | temp. | 0.061 | **0.004** | 0.015 | 0.022 | **0.020** | 0.081 | **0.074** | 0.003 | 0.006 |
| ToxicChat (jailbreak) | temp. | 0.079 | **0.005** | 0.031 | **0.005** | 0.012 | **0.023** | 0.062 | 0.001 | 0.001 |
| Banking77 | temp. | 0.043 | **0.027** | 0.086 | **0.206** | 0.309 | **0.548** | 1.234 | 0.027 | 0.051 |
| CLINC150 | – | 0.120 | 0.120 | **0.024** | 0.189 | **0.163** | **0.487** | 0.516 | 0.113 | 0.044 |
| AG News | temp. | 0.060 | **0.012** | 0.077 | **0.165** | 0.191 | **0.336** | 0.782 | 0.012 | 0.016 |
| Financial PhraseBank | – | 0.054 | **0.054** | 0.137 | **0.274** | 0.383 | **0.457** | 0.703 | 0.035 | 0.017 |
| Emotion | temp. | 0.103 | **0.026** | 0.279 | **0.295** | 0.672 | **0.636** | 2.831 | 0.026 | 0.042 |
| STS-B | temp. | 0.054 | **0.034** | 0.136 | **0.570** | 0.637 | **1.029** | 1.352 | 0.032 | 0.043 |
| SST-5 | temp. | 0.114 | **0.034** | 0.170 | **0.549** | 0.591 | **0.965** | 1.375 | 0.034 | 0.057 |
| ToxiGen (toxic) | Platt | 0.034 | 0.032 | **0.014** | 0.110 | **0.087** | 0.359 | **0.292** | 0.020 | 0.022 |
| ToxiGen (toxicity) | – | 0.120 | **0.120** | 0.200 | 0.680 | **0.664** | **1.316** | 1.592 | 0.084 | 0.050 |

Table 4: Calibration on the test split, our served model (batch size 1) against Jev on the same examples. The calibrator is chosen and fitted on the 200 gold dev examples only (5-fold CV negative log-likelihood) and frozen before a single logged test scoring; Choice and Score heads may only use argmax-preserving calibrators, so no decision changes. Score heads are scored as a distribution over levels (gold rounded to the nearest level). Bold is the better of the two systems. Ours is better on 12/14 heads in ECE, 10/14 in Brier score and 12/14 in NLL. *Oracle* columns recalibrate each system by 5-fold cross-validation on the test labels with the same allowed methods: what any post-hoc recalibration of either system could reach at best.

The practical lesson is to evaluate a dynamically quantized model the way it will be served, or to use static quantization when outputs must not depend on the batch.

**Robustness to duplication.** Some datasets repeat inputs across their own splits, and a model trained on the pool may have seen a test input's text (never its gold label). Table 6 re-scores the frozen predictions with exact duplicates removed.

- All eight wins hold, with similar margins.
- ToxicChat (6% duplicates) and ToxiGen (19%) lean towards Jev on the de-duplicated subsets.

Those two ties should be read as "at best on par".

## 6 What mattered and what did not

The 318 ledger entries let us separate the levers that moved the dev metric from those that did not. All numbers in this section are dev numbers (200 examples) unless stated otherwise.

**The labeling prompt was the largest lever.** Figure 4 shows labeler agreement with gold before and after the v2 prompt. Guidelines written from 20 seeds and the labeler's dev errors raised agreement by:

- 30 points on Emotion (0.49 → 0.785). The guideline encodes the dataset's convention that the label is the emotion word attached to "feel", regardless of negation.

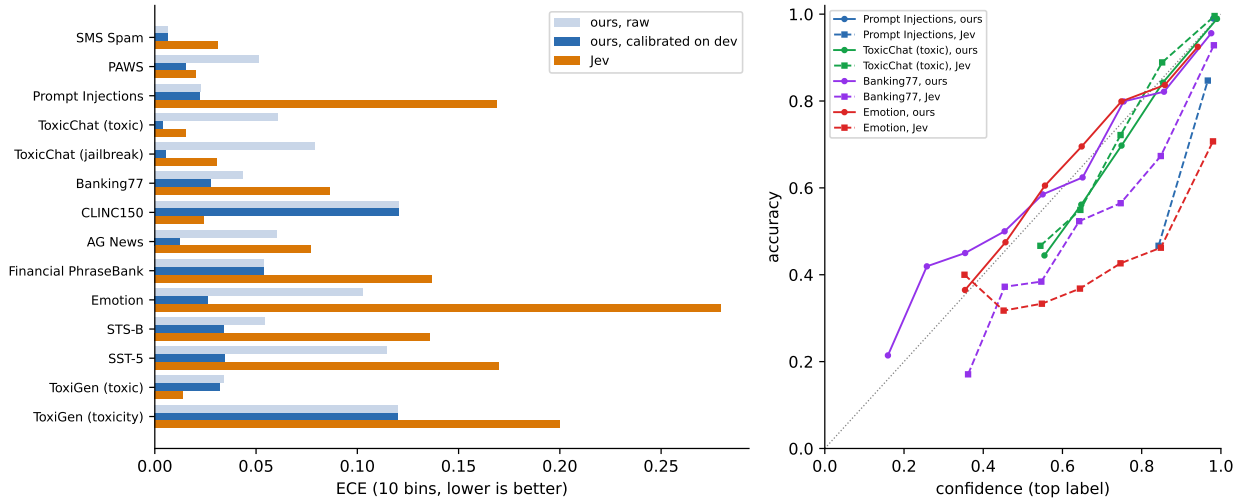



Figure 3: Left: expected calibration error on the test split, our served model before and after a calibrator fitted on 200 dev examples, against Jev on the same examples. Right: reliability diagrams (bins with $\geq$10 examples) for four heads; the dotted diagonal is perfect calibration.

- 10 points on Banking77 (0.735 $\rightarrow$ 0.84). The guideline adds descriptions for 77 intents that the question template leaves blank.
- 9 points on SST-5. Without guidance the labeler almost never chose "very positive".
- 6–8 points on ToxicChat, Prompt Injections, CLINC150 and STS-B.

On the half of dev never used to write the guidelines the gains are smaller but persist (Emotion: 0.60 with seeds only $\rightarrow$ 0.73 with guidelines). Writing guidelines from errors on dev can also overfit: a second SST-5 guideline gained 4 points on the half it was written from and lost 5 on the other.

**Stronger labelers rarely helped.** For SST-5 we probed nine labelers, from $0.03-per-million-token models up to frontier models, on the held-out dev half.

- All agreed with gold at 0.52–0.60, against 0.57 for the cheap labeler.
- A 5-labeler average reached 0.62, with a paired CI that included zero.
- Enabling reasoning did not help.

On Banking77, four larger models matched but did not beat a cheap labeler with good guidelines. On ToxicChat, a frontier labeler ranked prompts best (AP 0.883) but did not improve F1.

**Better labels do not always make a better student.** On ToxiGen, a 4-labeler ensemble raised labeler Spearman from 0.754 to 0.818, but the student moved only from 0.770 to 0.776. Students tend to plateau at or slightly below their labels, which makes the labeler's agreement with gold a cheap early predictor of the attainable score.

**Data: the whole pool, and synthetic data only for gaps.**

- **CLINC150:** labeling all 15k utterances instead of the exploratory round's 4.5k was the main change behind its test accuracy rising from 0.846 to 0.934. LLM-written out-of-scope utterances raised out-of-scope recall on dev from 0.81 to 0.935 at no cost to in-scope accuracy.
- **STS-B:** re-pairing pool sentences with their nearest neighbors, spread across the similarity scale and scored by the labeler ensemble, raised the seed-mean dev Spearman from 0.897 to 0.909. LLM-written pairs skewed to high similarity and did not help.

| Task | Batched | Batch 1 [95% CI] | Jev | Verdict | Flips | max $\lvert\Delta p\rvert$ |
|---|---|---|---|---|---|---|
| SMS Spam | 0.958 | **0.957** [0.946, 0.968] | 0.937 | win | 0.3% | 0.41 |
| PAWS | 0.884 | **0.881** [0.874, 0.888] | 0.850 | win | 1.3% | 0.63 |
| Prompt Injections | 0.957 | **0.966** [0.931, 0.991] | 0.741 | win | 0.9% | 0.20 |
| Banking77 | 0.861 | **0.871** [0.859, 0.883] | 0.797 | win | 3.7% | 0.76 |
| CLINC150 | 0.934 | **0.934** [0.927, 0.941] | 0.895 | win | 0.3% | 0.28 |
| AG News | 0.893 | **0.893** [0.886, 0.900] | 0.885 | win | 1.5% | 0.54 |
| Financial PhraseBank | 0.794 | **0.818** [0.792, 0.842] | 0.731 | win | 9.5% | 0.72 |
| SST-5 | 0.583 | 0.580 [0.559, 0.600] | 0.579 | tie | 5.1% | 0.25 |

Table 5: Serving consistency of the int8 models. Dynamic int8 quantization computes activation scales over the whole input batch, so a text's probabilities depend on its batch-mates. The test predictions of these tasks had been computed in batches; here they are recomputed one request at a time, the way the model is served and its latency measured, with the same artifact. *Flips*: share of test decisions that change. Every verdict is unchanged. ToxicChat and STS-B were already predicted at batch size 1; Emotion and ToxiGen run in fp32.

- **PAWS:** two teachers pseudo-labeled the 41k unlabeled pool pairs, and a student trained on the soft pseudo-labels reached 0.890 on dev.

**Bigger models did not help within the latency gate.** With the user's approval we relaxed the gate to 8 threads and allowed ∼400M backbones.

- ModernBERT-large [20] matched but did not beat the 150M student on ToxiGen, SST-5 and STS-B.
- It exceeded 70 ms on all but the shortest inputs.
- Eight threads gave only a ∼1.3× speed-up for a single model.

No task was re-scored under the relaxed setting.

**Training and serving details that mattered.**

- **Weight soups:** averaging fine-tunes that share an initialization worked, while averaging independently seeded runs scored below its members (STS-B).
- **Single-run selection:** on 200-example dev sets, seed variance often exceeded the effects being measured (four STS-B seeds of one recipe spanned 0.895–0.908), so best-of-seeds selection is optimistic.
- **int8 export:** plain int8 collapsed UltraBERT (accuracy 0.82 → 0.64). Keeping the MLP output projections in fp32, or using SmoothQuant rescaling, removed the loss.

# 7 A recipe for practitioners

1. Write the question exactly as you would for Jev. Hand-label about 220 examples: 20 seeds and 200 dev.
2. With a cheap LLM, label the seeds-plus-dev set; read the errors; write guidelines; re-check on the half of dev you did not read. Stop when agreement stops rising. This number predicts your ceiling.
3. Label your whole unlabeled pool in batches (roughly $0.10–$0.30 per 10k texts with a flash-class model). Average two cheap labelers if they disagree often.

| Task | Dup. removed | Ours | Jev | Δ |
|---|---|---|---|---|
| SMS Spam | 82/5354 | 0.957 | 0.936 | +0.022 |
| PAWS | 0/8000 | 0.884 | 0.850 | +0.035 |
| Prompt Injections | 0/116 | 0.957 | 0.741 | +0.216 |
| ToxicChat | 307/5083 | 0.775 | 0.790 | -0.015 |
| Banking77 | 25/3076 | 0.860 | 0.796 | +0.064 |
| CLINC150 | 19/5500 | 0.934 | 0.895 | +0.039 |
| AG News | 15/7600 | 0.893 | 0.885 | +0.008 |
| Financial PhraseBank | 4/970 | 0.795 | 0.732 | +0.063 |
| Emotion | 11/2000 | 0.814 | 0.587 | +0.228 |
| STS-B | 7/1379 | 0.893 | 0.890 | +0.003 |
| SST-5 | 2/2210 | 0.583 | 0.579 | +0.004 |
| ToxiGen | 177/940 | 0.845 | 0.879 | -0.034 |

Table 6: Robustness to train/test duplication. Frozen predictions re-scored after removing test inputs that exactly match (after normalisation) any seed, dev, or training-pool text the experiment could see; Jev re-scored on the same subset. All eight wins hold; the ToxicChat and ToxiGen ties lean towards Jev.

4. Add synthetic data only for gaps such as rare classes, out-of-scope inputs, or under-covered similarity levels.
5. Fine-tune a 150M encoder. Average 3 shared-init seeds. Fit the decision threshold on held-out data.
6. Calibrate on the 200 dev examples with `ownyourjev calibrate`: temperature for Choice and Score heads; temperature, Platt or isotonic for Noul heads, chosen by cross-validated log-loss.
7. Export to ONNX int8, keeping the MLP output projections in fp32. Measure CPU p95 on your hardware, and evaluate at the batch size you will serve.

## 8 Limitations

- **Scope.** We report the 12 classification and similarity tasks. Reading comprehension and judging generated text are future work.
- **Ties are not wins.** Four tasks are statistical ties. On ToxicChat and ToxiGen, duplication between the pool and test favors our models.
- **Test looks.** Two tasks were scored twice under the dev-gated policy. For STS-B the dev gain held on test (+0.015 on dev vs. +0.014 on test). For ToxicChat the second look was a latency fix that left accuracy unchanged. On tasks outside this paper, dev-gated gains shrank substantially on test, so small dev sets remain a real risk.
- **Human labels.** We use 220 human labels per task, while Jev is zero-shot. This is the intended comparison, since a practitioner can label 220 examples in an afternoon, but it is not label-free.
- **Version and latency.** We compare with `jev-1.13.0` (the published responses), and Jev's reported latency includes the network. Our latency is hardware-specific, and the cost figures depend on stated price assumptions.
- **Quantization.** Dynamic int8 outputs depend on the batch. We re-scored all int8 tasks at batch size 1 (no verdict changed), but on Financial PhraseBank 9.5% of decisions are sensitive to it, so quantization-aware training would make these models more robust.
- **Calibration.** Calibrators are fitted on 200 dev examples, too few to see rare regions such as CLINC150's out-of-scope class. Jev remains better calibrated on three heads.

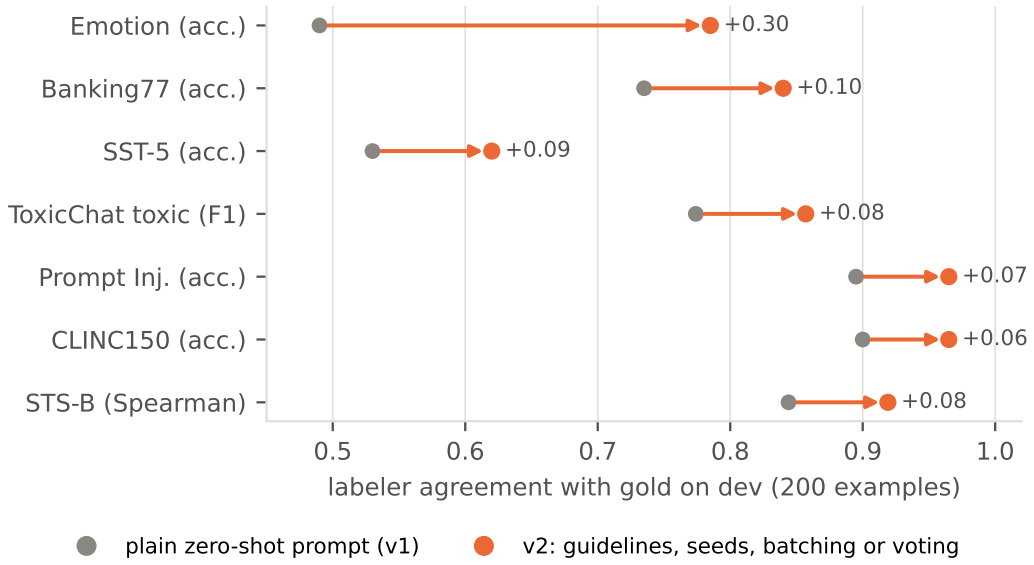



Figure 4: Labeler agreement with gold on dev before (plain zero-shot prompt) and after the v2 prompt (guidelines from seeds and dev errors, with few-shot seeds, batching, or labeler voting depending on the task). Numbers are taken from the experiment ledgers.

- **Label bias.** LLM labels inherit LLM biases. We measured agreement with gold but did not audit subgroup behavior.
- **AI-run experiments.** Experiments were run by coding agents under a fixed protocol. Spend caps, test access and every experiment are logged in the released ledgers. An exploratory precursor round that violated the protocol was discarded.

## 9 Conclusion

For the decisions most teams actually need, such as routing, topic, domain sentiment, spam, prompt injection, paraphrase and similarity, you do not need a hosted decision model. A cheap LLM, an afternoon spent labeling 220 examples, and a 150M encoder on a CPU match or beat Jev on all 12 tasks we report. The owned model's probabilities are better calibrated on 12 of 14 heads. It answers faster than Jev's best case, costs 12–84× less per decision, and recovers its build cost within thousands to a few hundred thousand decisions. It runs where your data already is and changes only when you retrain it. The code, the per-task ledgers and the audited test log are available at https://github.com/erfaneshrati/ownyourjev.

## References

[1] Tiago A. Almeida, José María Gómez Hidalgo, and Akebo Yamakami. Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering*, 2011.

[2] Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient

intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on NLP for Conversational AI*, 2020.

[3] Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of SemEval*, 2017.

[4] deepset. Prompt injections dataset. https://huggingface.co/datasets/deepset/prompt-injections, 2023.

[5] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. Evaluating and benchmarking the system one model Jev. *arXiv preprint arXiv:2609.37647*, 2026.

[6] DMIS Lab. UltraBERT: One encoder for classification and retrieval. https://huggingface.co/blog/dmis-lab/ultrabert, 2026.

[7] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30), 2023.

[8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, 2017.

[9] Thomas Hartvigsen, Saadia Gabriel, Hamid Palangi, Maarten Sap, Dipankar Ray, and Ece Kamar. ToxiGen: A large-scale machine-generated dataset for adversarial and implicit hate speech detection. In *Proceedings of ACL*, 2022.

[10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[11] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of EMNLP-IJCNLP*, 2019.

[12] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.

[13] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. ToxicChat: Unveiling hidden challenges of toxicity detection in real-world user-AI conversation. In *Findings of EMNLP*, 2023.

[14] Pekka Malo, Ankur Sinha, Pekka Korhonen, Jyrki Wallenius, and Pyry Takala. Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4), 2014.

[15] ONNX Runtime developers. ONNX Runtime. https://onnxruntime.ai, 2021.

[16] Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of EMNLP*, 2018.

[17] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, 2013.

[18] TypeSafe AI. Jev: a system one decision model. https://openrouter.ai/docs/guides/community/jev, 2026. Accessed 2026-10-17.

[19] Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. Want to reduce labeling cost? GPT-3 can help. In *Findings of EMNLP*, 2021.

[20] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[21] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of ICML*, 2022.

[22] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of ICML*, 2023.

[23] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, 2015.

[24] Yuan Zhang, Jason Baldridge, and Luheng He. PAWS: Paraphrase adversaries from word scrambling. In *Proceedings of NAACL-HLT*, 2019.

# A Per-task recipes

All students are fine-tuned end-to-end. "Soft" denotes training on per-option probabilities.

- **SMS Spam:** gte-modernbert-base, 5-fold cross-fit; deepseek-flash labels with seed examples; int8.
- **PAWS:** UltraBERT-150M; flash labels with seeds plus soft pseudo-labels on 41k unlabeled pairs from two UltraBERT teachers; partial int8 (attention matmuls and `down_proj` kept in fp32).
- **Prompt Injections:** gte-modernbert-base; 326 pool texts labeled with seed-derived guidelines (labeler dev accuracy 0.965), 1,148 seed-steered synthetic EN/DE inputs, and the 220 gold examples; int8.
- **ToxicChat:** one gte-modernbert encoder with two heads; soft labels averaged from two cheap labelers, plus synthetic hard positives and negatives; uniform soup of 7 shared-init runs; eager-attention export with SmoothQuant; thresholds from dev.
- **Banking77:** gte-modernbert; soft vote labels from four labelers (two on the whole pool, two adjudicating 840 disagreements), using a prompt with hand-written intent descriptions and all 220 gold examples; int8.
- **CLINC150:** gte-modernbert; the whole 15k pool labeled in batches of 20 with intent descriptions; 2,066 synthetic out-of-scope utterances; out-of-scope threshold 0.4; int8 on the MLP input projections only.
- **AG News:** gte-modernbert; 5k flash labels; int8.
- **Financial PhraseBank:** gte-modernbert; 3.6k flash labels with seeds; int8.

- **Emotion:** gte-modernbert; 10k flash labels with the "feel-word" guideline and seeds; fp32 ONNX (int8 cost 2 dev points).
- **STS-B:** gte-modernbert; 4-labeler averaged soft scores with SemEval guidance, sentence-swap augmentation, 7.6k re-paired pool pairs, and a 3-run shared-init soup; int8 with the MLP output projections kept in fp32.
- **SST-5:** UltraBERT-150M; soft labels with SST-convention guidance and seeds; partial int8.
- **ToxiGen:** one gte-modernbert encoder with two heads; soft labels for $P(\text{toxic})$ and a 5-level distribution; *toxic* derived as expected level $/4 \geq 0.30$ (chosen by CV on dev); fp32 ONNX.

# B Test-access log

Table 7 lists every test scoring of the 12 reported tasks. Both re-scorings carry logged evidence:

- STS-B: 7.6k re-paired pairs plus a shared-init soup; dev $+0.015$, CI $[+0.004, +0.028]$.
- ToxicChat: an idle-machine re-measurement showed the first model exceeded the latency gate (p95 68–75 ms). The fix (eager export plus SmoothQuant) left dev F1 unchanged.

The full log, including the deferred tasks, is in `benchmarks/results_v2/TEST_ACCESS.log`.

| Time | Task | Ours | Jev | Kind |
|---|---|---|---|---|
| 2026-10-18 19:52 | SMS Spam | 0.9580 | 0.9366 | first scoring |
| 2026-10-18 20:00 | Financial PhraseBank | 0.7938 | 0.7309 | first scoring |
| 2026-10-18 20:14 | AG News | 0.8930 | 0.8850 | first scoring |
| 2026-10-18 20:16 | Banking77 | 0.8609 | 0.7975 | first scoring |
| 2026-10-18 20:49 | CLINC150 | 0.9340 | 0.8949 | first scoring |
| 2026-10-18 21:47 | STS-B | 0.8791 | 0.8900 | first scoring |
| 2026-10-18 22:04 | ToxiGen | 0.8543 | 0.8777 | first scoring |
| 2026-10-18 22:16 | Emotion | 0.8115 | 0.5850 | first scoring |
| 2026-10-18 22:31 | SST-5 | 0.5828 | 0.5792 | first scoring |
| 2026-10-18 23:53 | PAWS | 0.8845 | 0.8499 | first scoring |
| 2026-10-18 23:55 | Prompt Injections | 0.9569 | 0.7414 | first scoring |
| 2026-10-19 01:11 | ToxicChat | 0.7875 | 0.7862 | first scoring |
| 2026-10-19 01:31 | STS-B | 0.8935 | 0.8900 | re-score (dev-gated) |
| 2026-10-19 02:59 | ToxicChat | 0.7870 | 0.7862 | re-score (latency gate) |

Table 7: Every test scoring of the 12 reported tasks, from the audited log. The log also records 23 analysis-only accesses (calibration, batch-1 serving, oracle recompute, de-duplication) (Section 5), none of which changed a model, a threshold or a decision.